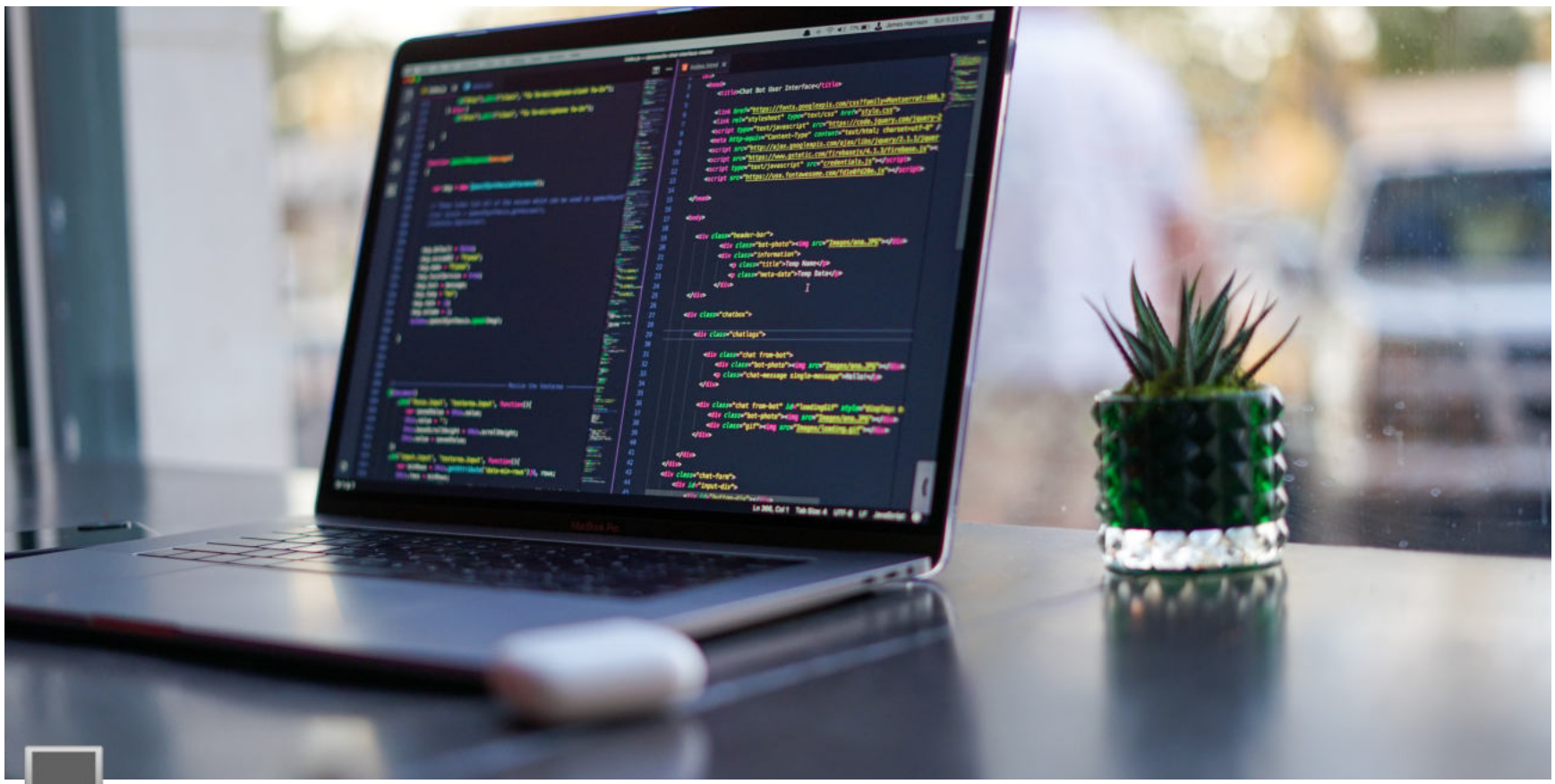

Week 8 Lecture 1

Class	BSCCS2003
Created	@October 24, 2021 12:43 AM
Materials	
Module #	44
Type	Lecture
Week #	8

Application Frontends

What is an application frontend?

- User-facing interface
  - General GUI application on the desktop
  - Browser based client
  - Custom embedded interface
- Device/OS specific controls and interfaces
- Web browser standardization
  - Common conventions among multiple browsers on how to render, what to render
- Browser vs Native
  - Look and feel
  - APIs, interfaces and interactions

Web Applications

- Browser based → HTML + CSS + JavaScript
  - HTML → What to show
  - CSS → How to show it
  - JavaScript → Bonus interaction (not core UI but essential for dynamic experience)
- Frontend mechanisms?

    - How to generate the HTML, CSS and JS?
    - Functional re-use, common frameworks
    - Server/Client load implications
    - Security implications

Fully static pages

- All (or most) pages on the site are statically generated
    - Compiled ahead of time
    - Not generated at run-time
- Excellent for high performance
    - Server just picks up the file and delivers
- How do you adapt to the runtime conditions?
    - User login, user specific information, time of the day
    - JavaScript can help a lot
- Increasingly popular → Static site generators
    - Jekyll, Hugo, Next.js, Gatsby
    - JavaScript allows very interesting variants

Run-time HTML generation

- Traditional CGI/WSGI based apps
    - Python (Flask, Django, ...), Ruby (Ruby on Rails)
    - PHPs core concept → Server-side run-time generation of HTML
    - WordPress, Drupal, Joomla → traditional CMS applications
- Great flexibility
    - Common layouts, adaptation and theming easy
    - Run-time changes, user login, time of the day, etc
- Server load
    - Every page has to be generated dynamically
    - May involve databases hits
    - Cost
    - SPeed
- Caching and other technologies can help, but complex

Client Load

- Typical Web browser
    - issue requests, wait for responses
    - render HTML
    - wait for the user input → most time spent waiting here
- Why not let client do more?
    - Allows more fancy interactions
- Client-side scripting
    - JavaScript de-factor standard
    - Component frameworks allows reuse, complex interactions
    - Server side JavaScript Node.js

Tradeoffs

- Server-side rendering
  - Very flexible
  - May be easier to develop
  - Less security issues on client
  - Load on the server
  - More security issues on the server
- Static
  - Cache-friendly
  - Very fast
  - Interaction difficult/impossible?
  - Compilation phase → small changes require compilation
- Client-side
  - Can combine well with static pages
  - Less load on the server but still dynamic
  - More resources needed on client
  - Potential security issues, data leakage

Estimating performance

https://server.guy.com/comparison/apache-vs-nginx/

- Static pages:
  - Apache ~ 10,000 req/s - 512 parallel requests
  - Nginx ~ 20,000 req/s - 512 parallel requests
- Dynamic (call out to PHP - limited by page rendering in PHP)
  - Both ~ 100 req/s @ 16 parallel
- Dynamic occupies more resources for longer → harder to scale
- Severe impact on the server